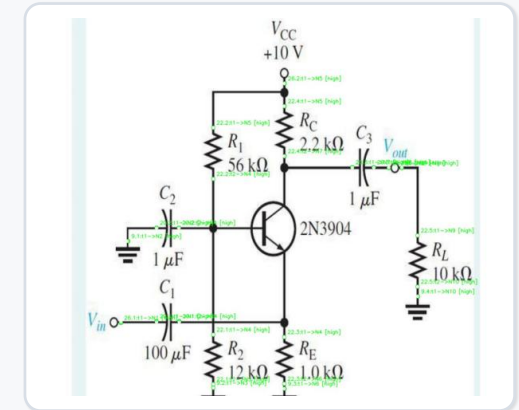
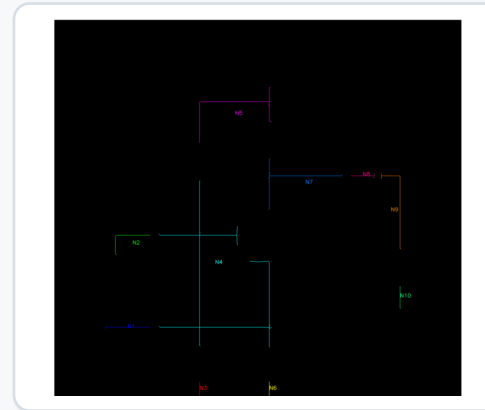
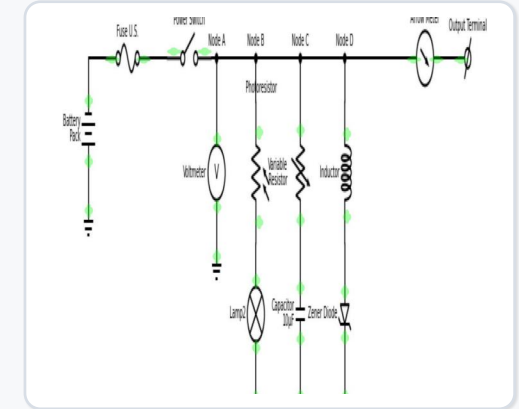
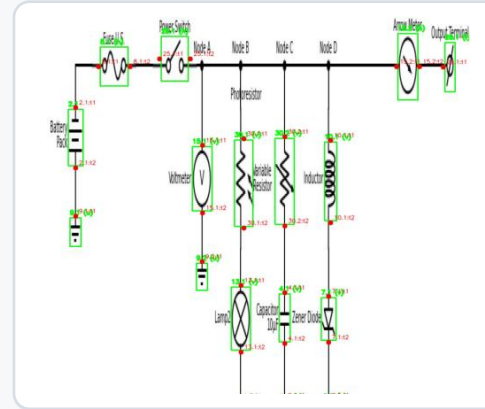


Pipeline topologica batch v1

Sintesi delle modifiche introdotte oggi negli script 01-07, dal confronto con la pipeline pilot fino all'export finale per graph database.

Focus del deck

- 01-02: invarianti rispetto al pilot
- 03-06: cambiamenti algoritmici e debug visivo
- 07: export grafo e significato dei CSV



Dal bbox alla topologia esportabile

estimate terminals → extract wires → build nets → match → export graph

Panoramica della pipeline

Dove la batch v1 resta invariata e dove cambia davvero

L'architettura generale è la stessa del pilot. Le modifiche sostanziali si concentrano nei passi 03-06, cioè nella parte che traduce detection e skeleton in connessioni affidabili.



Passi quasi invariati

- 01 detect_components: stessa inferenza YOLO e stessa struttura del JSON
- 02 assign_instances: stessa logica di ordinamento e assegnazione degli instance_id
- Le differenze sono operative, non algoritmiche

Passi che cambiano il comportamento

- 03: stima terminali image-aware e strategie per famiglie di simboli
- 04: preservazione direzionata del contatto locale terminale-wire
- 05-06: snap, preferred net, multi-stage search, confidence
- 07: export batch-ready per graph database e CSV aggregati

01-02: base invariata rispetto al pilot

Il batch v1 parte da una detection equivalente e mantiene lo stesso meccanismo di assegnazione delle istanze

01_detect_components

- stesso caricamento del modello YOLO e stessi metadati dal YAML
- stessi parametri principali di inferenza (imgsz, conf, iou)
- stesso JSON per immagine con classi rilevabili, classi per terminali e masking
- stessa immagine debug con bbox e label

02_assign_instances

- stesso ordinamento per class_id e SORT_ORDER
- stessa assegnazione degli instance_id nel formato class.index
- stesso riordino finale dei componenti e stessa immagine debug
- unica estensione utile: campo n_components nel JSON

03_estimate_terminals: da bbox-only a image-aware

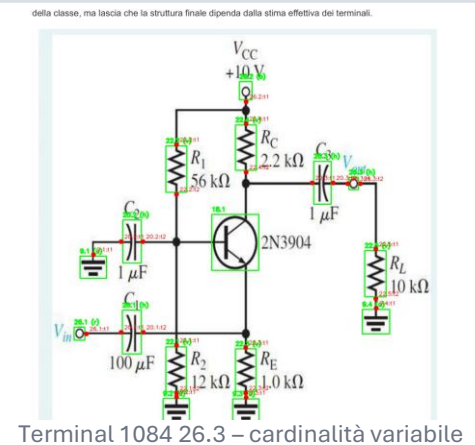
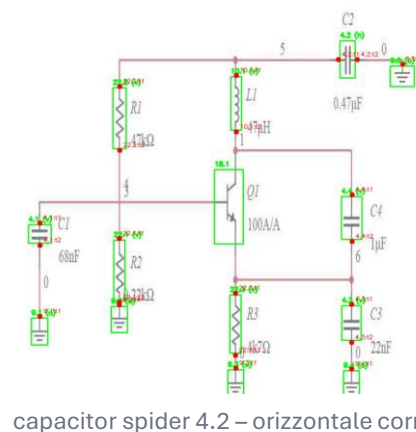
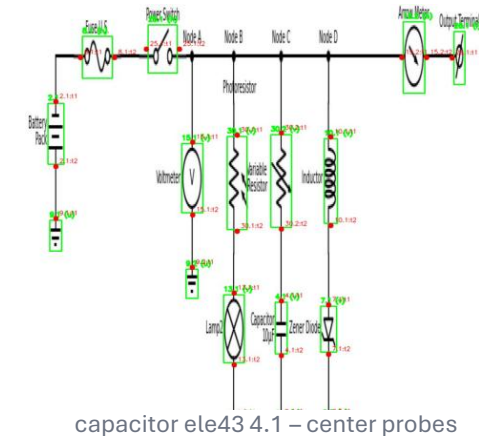
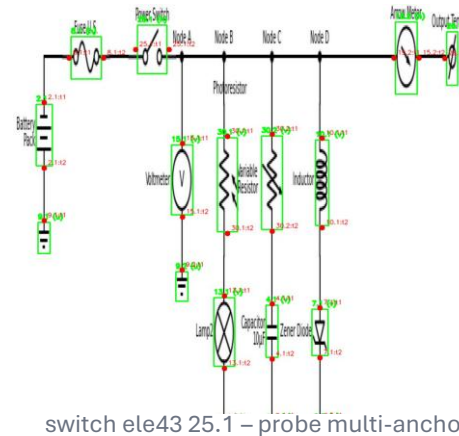
Il passaggio più importante della giornata: la stima dei terminali non dipende più solo dal bounding box

Cosa cambia davvero

- foreground binarizzato (grayscale + Otsu inversa) per leggere il contatto locale
- nuove strategie:
 - one_terminal_by_orientation,**
 - two_terminal_by_connection_axis,**
 - two_terminal_capacitor,**
 - two_terminal_switch,**
 - terminal_auto_one_or_two**
- probe locali sui candidati top / bottom / left / right
- separazione esplicita fra logica generica, capacitor e switch
- Terminal non più rigidamente monoconnesso: cardinalità derivata dalla stima effettiva

Messaggio chiave

La batch v1 usa metadata + bbox + contenuto locale dell'immagine.



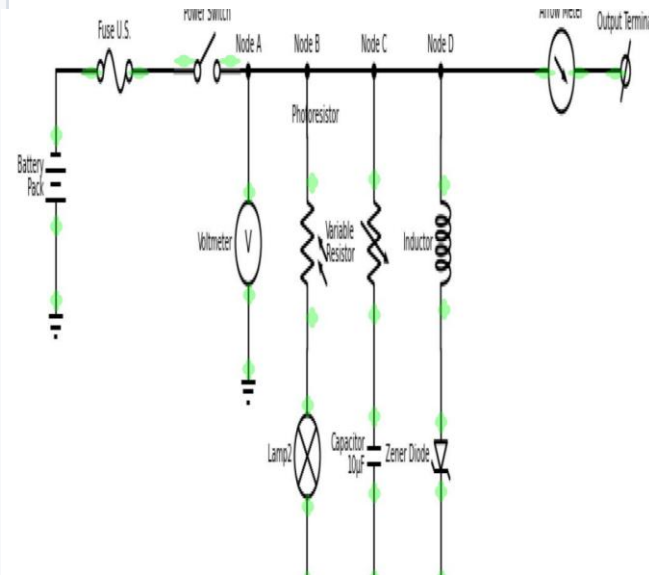
04_extract_wires: preservare il contatto, non solo il punto

La pipeline resta la stessa; cambia il modo in cui la maschera riapre le zone terminali

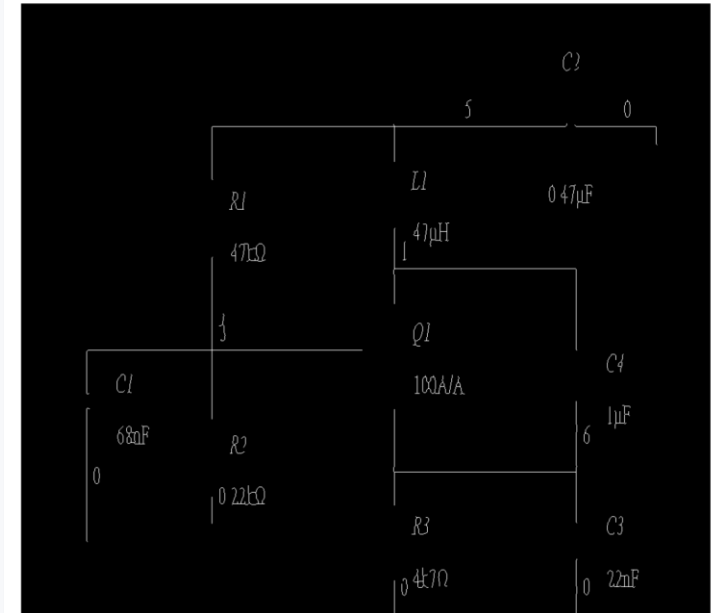
Evoluzione rispetto al pilot

- stessa sequenza generale: masking → Otsu → closing → filtro piccoli componenti → skeletonize
- la differenza è nel keep locale: da cerchio isotropo a cerchio + capsula direzionata
- la direzione deriva da relative_position del terminale (left/right = orizzontale, top/bottom = verticale)
- preservazione asimmetrica inward / outward per tollerare terminali non pixel-perfect
- terminal_keep_debug rende visibile dove la maschera è stata riaperta

Ruolo nel sistema: il passo 04 diventa il ponte robusto tra il 03 e i passi 05-06.



keep zone direzionate



effetto sul wire / skeleton

05_build_nets: stessa logica di net, match terminale più robusto

Le connected components restano la base delle net; migliora l'associazione locale terminale → componente

Pilot

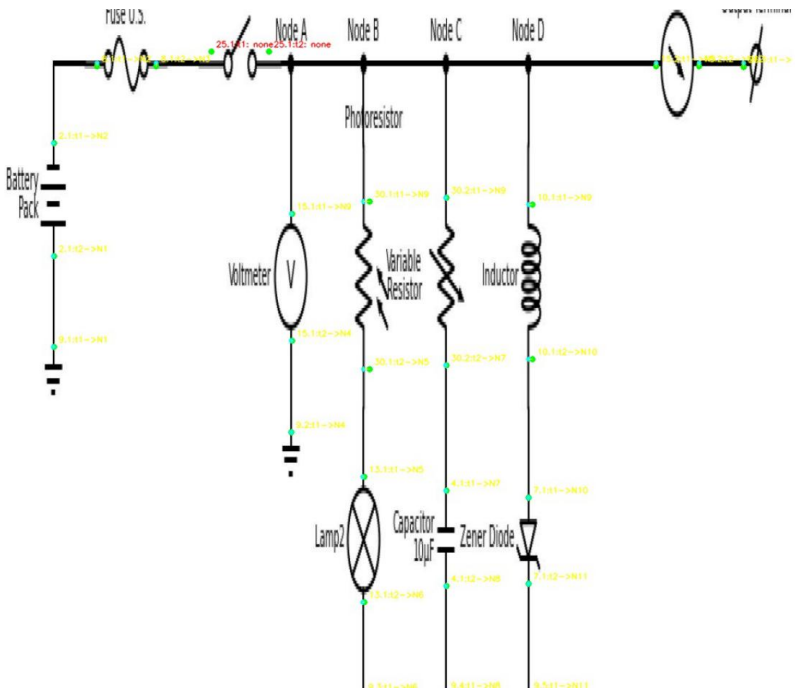
finestra quadrata attorno al terminale

Batch v1

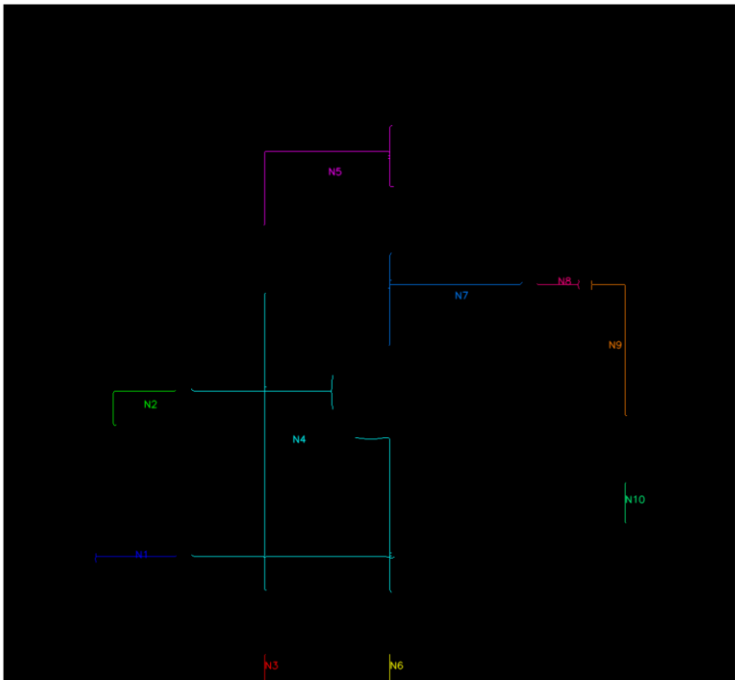
ricerca direzionale primaria + fallback quadrato + primary_label + snap_point

Output extra

terminal_debug e rejected_candidates



terminal_debug: terminale, snap sullo skeleton, net associata



net_map: connected components promosse a N1, N2, ...

06_match_terminals_to_nets: multi-stage search + confidence

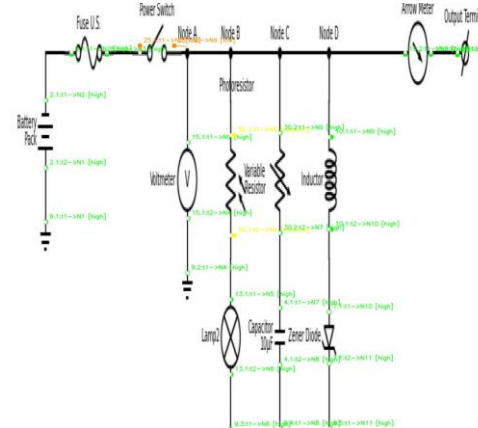
Il match finale non è più solo “più vicino”, ma un risultato spiegabile, graduato e robusto

Punti chiave

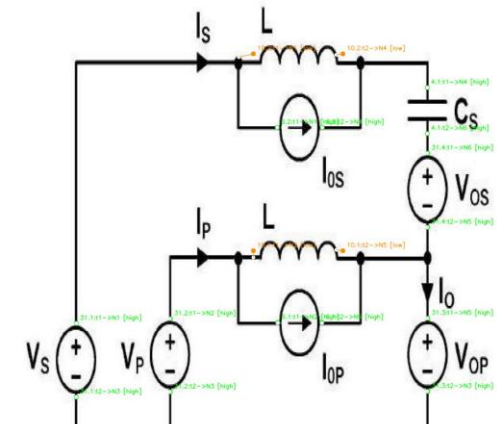
- usa la preferred net suggerita dal passo 05 quando disponibile
- piano di ricerca multi-stage: directional_primary → circle_primary → directional_fallback → circle_fallback
- override di ricerca per classi difficili (Switch, Inductor, Meter, Current/Voltage Source)
- stati di match più informativi: matched_preferred, matched_single, matched_nearest, unmatched
- confidence high / medium / low / none + warnings + flag is_suspicious_match

Colori debug

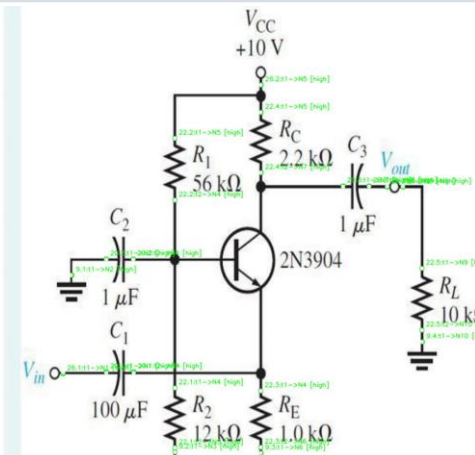
verde = high giallo = medium · arancione = low · rosso = unmatched



ele43 – switch con low confidence



1855 – induttori recuperati ma deboli

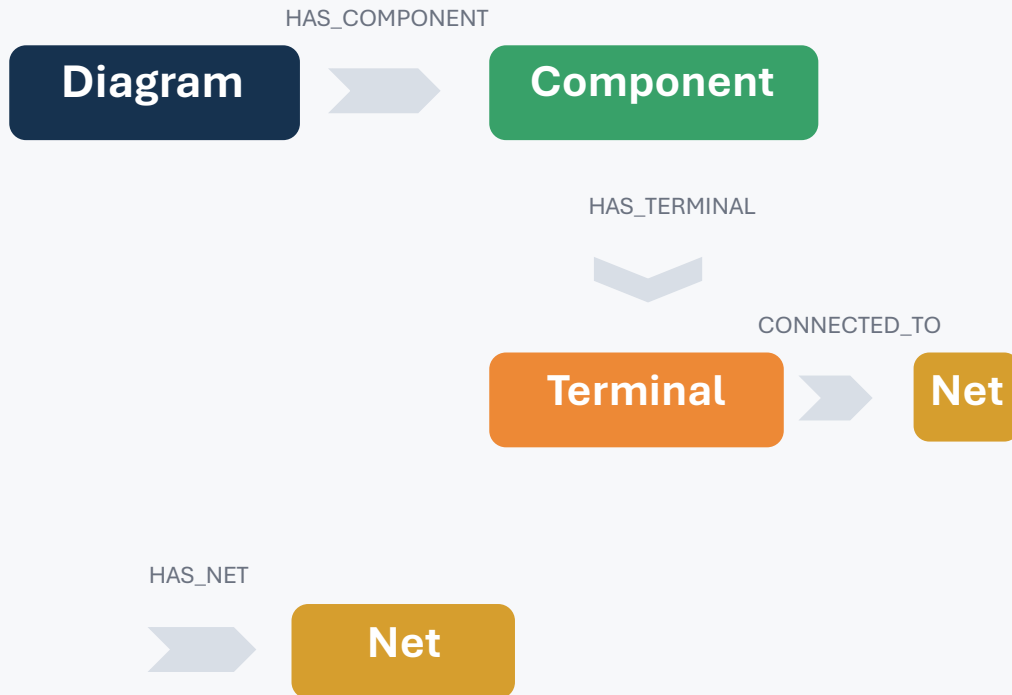


caso pulito ad alta confidence

07_export_graph: nodi, archi e CSV per il batch

Il grafo mantiene la stessa topologia concettuale del pilot, ma diventa batch-ready e più ricco di attributi

Modello esportato



ID univoci + diagnostica del matching rendono l'export pronto per il graph database.

Cosa descrivono i CSV

Per diagramma

- `nodes_csv/<diagramma>_nodes.csv` = tutte le entità del singolo grafo (Diagram, Component, Terminal, Net)
- `edges_csv/<diagramma>_edges.csv` = tutte le relazioni del singolo grafo (HAS_COMPONENT, HAS_NET, HAS_TERMINAL, CONNECTED_TO)

A livello batch

- `combined_csv/all_nodes.csv` = concatenazione di tutti i nodi di tutti i diagrammi
- `combined_csv/all_edges.csv` = concatenazione di tutti gli archi del batch
- `combined_csv/graph_summaries.csv` = un record per diagramma con metriche sintetiche del grafo e della confidence

Takeaways finali

Dove si vede il valore della batch v1 rispetto al pilot

1. Più aderenza alla geometria reale

- 03 e 04 smettono di assumere bbox e terminali pixel-perfect
- la pipeline legge meglio il contatto locale simbolo-wire
- capacitor e switch vengono finalmente trattati con logiche diverse

2. Più robustezza topologica

- 05 costruisce le net come prima, ma con `primary_label`, `snap_point` e `terminal_debug`
- 06 recupera casi difficili con ricerca multi-stage e class overrides
- i match deboli non vengono persi: vengono tracciati e graduati

3. Export pronto per il graph DB

- 07 esporta nodi e archi con ID univoci a livello batch
- i CSV descrivono sia il singolo diagramma sia il batch completo
- la diagnostica del matching entra direttamente nell'export finale

In sintesi: 01-02 sono una trasposizione batch del pilot; il vero salto di qualità è la catena 03 → 06, culminata nel 07 con un export batch-ready per il grafo.